

Plan Along the Way: Event-Triggered Foundation-Model Planning for TAMP Execution in Partially Observable Manipulation

Abstract—Manipulation in partially observable environments requires planning under incomplete scene information, revising plans as object interactions reveals previously hidden details. In such settings, an initially valid plan may execute successfully yet remain insufficient for task completion. For example, opening a container may reveal an object provides important context for the successful completion of the task. Existing foundation-model-guided task and motion planning (TAMP) systems can generate useful long-horizon task decompositions, subgoals, or constraints, but they often assume having access to a fully specified scene state or invoke model-level replanning after a subgoal, refinement, or execution attempt fails. We present ROBUST TAMP, a modular LLM/VLM-guided planning framework for reactive TAMP where unseen task-relevant and non-target objects may become visible during execution. The framework restricts the foundation-model planner to the currently visible relational scene state, validates generated task-level actions against a strict executable interface, and routes the accepted actions to scene-specific execution adapters. Object discovery is treated as a distinct replanning event and after a stable execution horizon, the system reconstructs the visible scene state and replans using completed-action history and structured replanning event context. Evaluations are performed on six RLBench/CoppeliaSim kitchen and grill variants involving hidden objects, non-target object discovery, articulated-container interaction, and temporal manipulation procedures. We compare text-only LLM and VLM planners of different sizes under the same validation, execution, monitoring, and replanning pipeline, reporting task success, partial goal completion, discovery- and failure-triggered replanning behavior, implicit non-target-object handling, and planner inference cost.

I. INTRODUCTION

Autonomous manipulation systems often have to reason and act under incomplete scene information. During long-horizon tasks, robots frequently encounter occluded or latent task-relevant objects whose importance is revealed only through interaction with the environment. In such settings, the planner must reason over the current scene information, while avoiding assumptions about unobserved objects. This creates a failure mode different from standard plan-execution failure: an initial plan may be valid for the visible scene and may execute successfully, yet remain insufficient for task completion as environment interaction reveals a previously hidden task-relevant object that affects the task.

Task and motion planning (TAMP) provides a principled way to interleave discrete task actions with appropriate continuous robot motions [1]. The coupling between task planning and motion planning is essential as a discrete action might be infeasible for the motion planner (e.g., picking an object obstructed by another object). TAMP planners classify an action as infeasible based on timeouts, wasting

time for infeasible motion planning problems. This may lead to multiple iterations of replanning, e.g, removing blocking objects, which is expensive. Additionally, TAMP planners require explicit symbolic domain specifications.

Recent work has therefore explored the use of large language models (LLMs) and vision-language models (VLMs) as semantic components within the planning systems. These models are able to translate natural-language goals into formal planning representations, generate symbolic task skeletons, infer task constraints, produce intermediate subgoals, and guide or repair TAMP search [2]–[10]. Further, open-state approaches maintain and update object attributes over long action histories, while scene-graph-based approaches use structured environment representations to ground large-scale task planning and support iterative replanning [11], [12]. These approaches demonstrate that foundation models (FM) can provide useful commonsense structure for long-horizon TAMP, reducing replanning.

Yet, these approaches assume a fully observable environment, generating plans solely from the currently observed scene. Consequently, they fail in partially observable environments by ignoring unobserved objects that may affect task execution. In contrast, our approach explicitly models multi-level failure modes under incomplete scene information, enabling the robot to reason about task-relevant objects that become observable through interactions with the environment. We treat object discovery as a first-class replanning event. When an action reveals an unobserved object, execution pauses only at a stable execution horizon. The visible scene state is then reconstructed, augmenting the planner with completed-action history and structured replanning event context, querying the FM to revise the remaining task sequence. Unrecovered execution failures follow the same replanning path. Recoverable geometric and execution failures, such as alternative grasps, or placements, are handled locally by the execution backend before escalation to the FM. This separates semantic replanning from executor-local recovery, reducing unnecessary FM calls.

We present ROBUST TAMP, a modular LLM/VLM-guided TAMP framework for long-horizon manipulation under partial observability. At each replanning, the system constructs a visible-object relational scene state from the current observation. An FM backend gives a task plan over this visible state with historical context. The generated action sequence is validated against a strict executable interface, allowing invalid actions to be detected before execution, and delegated to scene-specific execution adapters, grounding the valid actions using the manipulation backend.

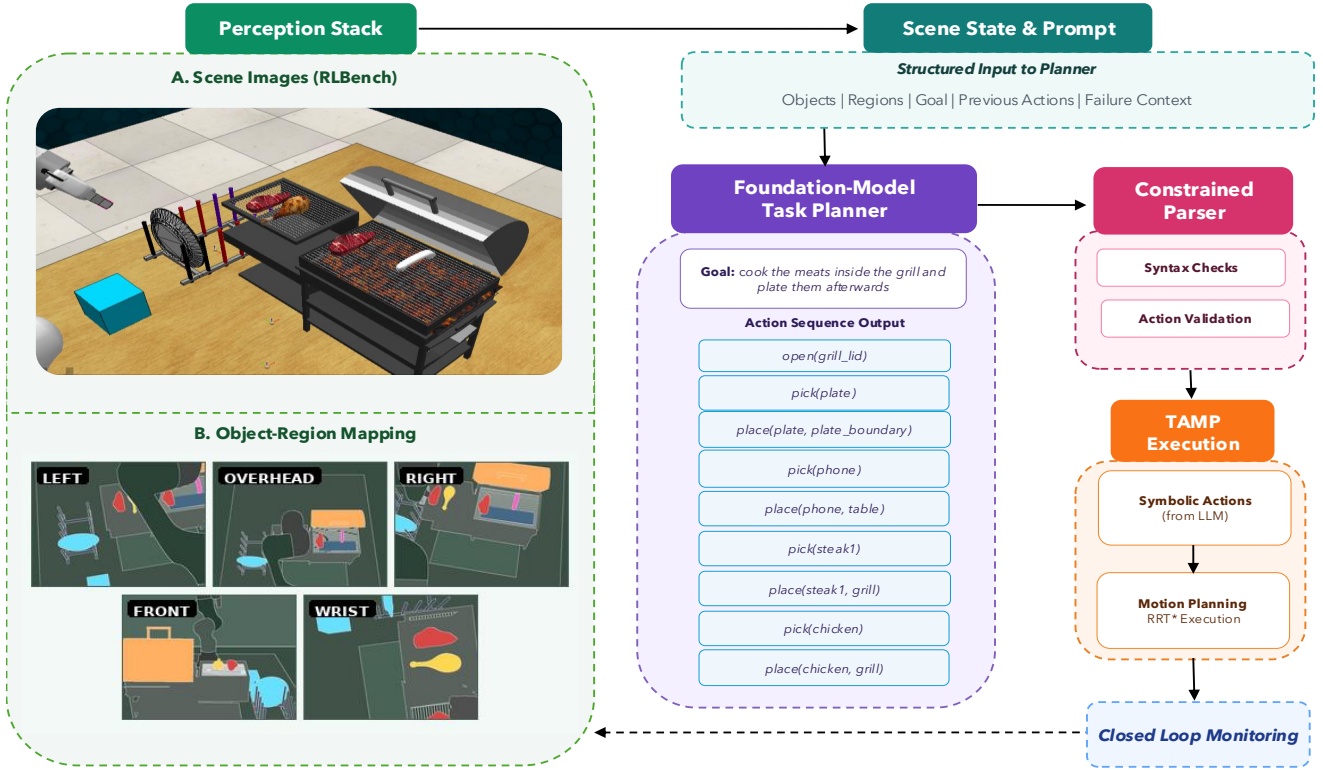


Fig. 1: Overview of ROBUST TAMP.

The primary contributions of this work are: (1) An FM-based task planner that leverages common sense and geometric reasoning from scenes to synthesize feasible actions, reducing overall TAMP replanning. (2) A visible-object-only planning formulation that prevents the FM planner from reasoning over unobserved scene objects, (3) Event-triggered replanning, which incorporates newly revealed scene objects and allows the FM planner to infer how the discovered object modifies the task requirements, and (4) Layered failure-escalation architecture that rejects structurally invalid plans before execution, handling recoverable failures within the execution backend, invoking the FM planner with structured failure context only when replanning is required.

II. RELATED WORK

Domain Specific and Procedural Task Encodings. Classical TAMP require domain specific abstractions (relevant objects, predicates, actions, goal conditions) which cannot encode all geometric details, causing some actions to be infeasible at motion-level [13], [14]. Recent LLM-based planning systems alleviate domain level specifications by translating natural-language tasks into formal planning representations. LLM+P translates natural-language planning problems into PDDL [2]. AutoTAMP uses LLMs to translate task specifications into formal constraints and check them autoregressively [3]. CaStL extracts goal, ordering, and blocking constraints from language and translates them into PDDL and Python specifications [4]. PRoC3S and LLM3 further use LLMs to propose symbolic plans or continuous parameters that

are checked by downstream constraint or motion-planning modules [6], [7]. These approaches show that procedural constraints can be represented in formal planning systems. However, doing so typically requires procedural states, object roles, action effects, and ordering constraints to be explicitly encoded in the domain or problem specification.

ROBUST TAMP targets a different setting: the FM planner is not given the complete object set, hidden object roles, final evaluation relations, or procedural predicates. It receives only the currently visible relational state and the natural-language goal. Procedural requirements, such as cooking an object before serving it, are verified by deterministic task evaluation, while the planner must infer the required sequence from the observed scene and goal text.

Failure-Triggered Replanning in FM-Guided TAMP. FMs have also been used to guide TAMP through high-level skeletons, subgoals, constraints, and visual grounding. OWL-TAMP uses VLM-inferred constraints to guide open-world TAMP [8]. VLM-TAMP uses VLM-generated intermediate subgoals to reduce the search horizon, re-quiring the VLM when a subgoal/action cannot be refined [9]. VIZ-COAST uses VLM reasoning to infer spatial constraints that reduce downward-refinement failures [10]. Related approaches use visual grounding, affordance prediction, or VLM-as-planner/grounder interfaces for planning [15]–[17].

Most approaches initiate replanning when the current plan, a subgoal, a refinement step, or an execution attempt fails. In contrast, ROBUST TAMP treats object discovery as a distinct replanning event rather than as an execution failure,

since previously hidden objects may influence the task. This enables discovery-triggered replanning and failure-triggered replanning to be analyzed separately.

Closed-loop Embodied Planning. Closed-loop embodied planning systems use execution feedback, precondition violations, or environmental observations to correct plans during execution. Inner Monologue incorporates environment feedback into language-model reasoning [18]. CAPE prompts LLMs with precondition errors to produce corrective actions [19]. DoReMi detects plan-execution misalignment with language or visual feedback for recovery [5]. Replan-VLM introduces internal and external correction mechanisms for replanning after task execution failures [20]. Reflective Planning uses test-time reflection and predicted future states to improve long-horizon manipulation [21]. Other systems update open-world state or structured scene representations during planning [11], [12], [22], [23].

Such representations introduce additional challenges: stale or incorrect state estimates may persist across replanning cycles, increasing the likelihood of hallucinations. ROBUST TAMP addresses these challenges through a more conservative state interface. It does not maintain a persistent planner-visible memory of hidden objects and does not expose inferred object roles before they are observed. Instead, at each replanning, the planner-visible state is reconstructed from current evidence, object-region relations, target regions, articulation state, completed actions, and structured event context. Recoverable grounding failures are handled locally by the execution backend before escalation to the FM planner. This design isolates the problem of visible-state replanning under object discovery, while still preserving validation of final object relations and procedural task completion.

III. PROBLEM FORMULATION

We consider long-horizon manipulation tasks specified by natural-language goals in environments where relevant scene objects may be initially hidden and become observable only through interaction. Newly revealed objects may be directly required to satisfy the goal, or may constitute non-target objects whose presence may affect task feasibility or execution. ROBUST TAMP plans using the currently available scene evidence, executes selected task-level actions using a manipulation backend, and invokes replanning only when a planning event occurs. Planning events are triggered at the initial state, upon the discovery of a previously unobserved object, or following an unrecovered execution failure.

A. Partial Observability and Visible Planning State

We denote by $\mathcal{O}$, the set of all objects o in a given scene. Goal-relevant objects will be denoted by $\mathcal{O}_g \subseteq \mathcal{O}$. The set of non-target objects, i.e., objects not referenced by the natural-language goal is therefore $\mathcal{O} \setminus \mathcal{O}_g$. A non-target object may obstruct an action or occupy a target region, and may require manipulation for the task completion. Under partial observability, the complete object set $\mathcal{O}$ is not assumed to be available to the FM planner. Instead, planning occurs at discrete planning events k , at which the planner receives

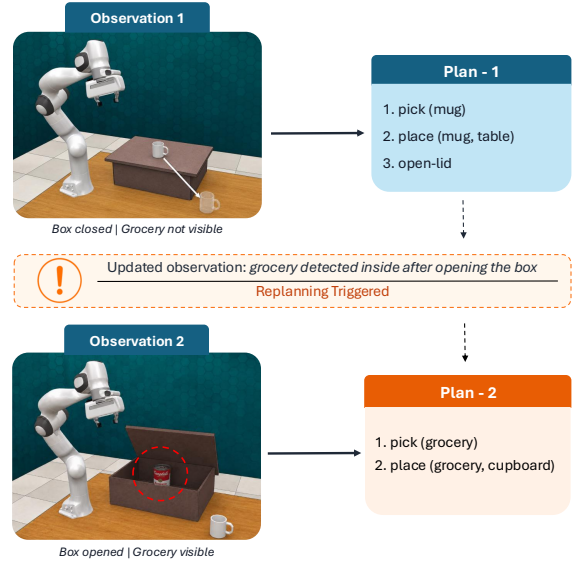


Fig. 2: Snippet of the Kitchen task. The robot must place mugs inside the container and groceries in the cupboard (not shown in the image). Observation 2— task-relevant object initially hidden.

evidence only about the currently visible object set $V_k \subseteq \mathcal{O}$. Objects in $\mathcal{O} \setminus V_k$, regardless of their relationship to the goal, are excluded from the planner’s scene evidence until they become observable through interaction.

At any planning event k , the planner-visible context is given by $c_k = (s_k^{\text{vis}}, h_k, e_k, g)$, where s_k^{vis} describes the currently visible scene, including V_k , their observed region assignments, available placement regions, robot state, and articulation or access states; h_k contains the actions executed until k ; e_k is a structured event context (e.g, a newly discovered object or an unrecovered execution failure); and g is the natural-language task goal. In Fig. 2, the initial set s_k^{vis} includes the visible mugs, groceries, container lid, cupboard, and placement regions. Thus, c_k allows later plans to incorporate newly available scene evidence without assuming knowledge of objects that remain unobserved or prescribing how newly revealed objects relate to the goal.

B. Planning and Execution Interface

At each planning event k , the FM planner maps the planner-visible context c_k to an ordered task-level action sequence $\alpha_k = \pi(c_k) = [a_1, a_2, \dots, a_n]$. Each action a_i belongs to the executable action set $\mathcal{A}_d$ specific to the domain d — for example in the kitchen domain, $\mathcal{A}_d = \{\text{pick}(o), \text{place}(o, r), \text{open}(c)\}$, where o denotes an object, r is a placement region, and c denotes a container. The generated sequence specifies the intended action order but does not directly specify grasp poses, trajectories, or other continuous execution parameters. Before execution, a validator checks whether α_k is realizable. If it is, a structured planning failure is generated; otherwise, the validated sequence is sent to the execution backend E_d . E_d grounds and attempts the actions using execution adapters $\mathcal{X}_d$, which report structured outcomes indicating successful execution or a failure requiring further system-level handling.

C. Task Objective

Given a task instance τ with a natural-language goal g_τ , the objective for the FM planner is to synthesize α_k and execute it across planning events until the task is completed or execution terminates. Let $\text{region}(s_k^{\text{vis}}, o)$ denote the semantic region occupied by object o at k . To verify the degree of task completion we measure two quantities, namely (1) a set of required final object-region relations $\mathcal{R}_\tau$, and (2) a set of required procedural predicates $\mathcal{P}_\tau$. Task completion can now be defined as

$$G_\tau(s_k^{\text{vis}}, h_k) = \bigwedge_{(o,r) \in \mathcal{R}_\tau} [\text{region}(s_k^{\text{vis}}, o) = r] \wedge \bigwedge_{p \in \mathcal{P}_\tau} p(h_k)$$

where $p \in \mathcal{P}_\tau$ is a deterministic predicate over h_k that verifies a required temporal procedure. For the kitchen tasks, $\mathcal{R}_\tau$ specifies the goal region for mugs and groceries, and $\mathcal{P}_\tau$ is empty. For the grill tasks (Fig. 1 A; see Section V-A for more details), $\mathcal{R}_\tau$ specifies the goal regions for plates and meats; $\mathcal{P}_\tau$ verifies cooking procedures that cannot be established from the final scene alone, such as placing a raw meat inside the grill, closing and reopening the grill, and subsequently placing the meat on the plate.

IV. METHODOLOGY

An overview of ROBUST TAMP is given in Fig. 1. At each planning event k , the framework constructs a planner-visible scene state s_k^{vis} , queries an LLM/VLM for a task-level action sequence α_k , validates it, and sends it to the execution layer E_d . E_d reports structured outcomes indicating successful execution or a failure requiring further planning iterations through execution adapter $\mathcal{X}_d$. The methodology is summarized in Algorithm 1.

A. Visible Scene-State Construction

At every planning event k , s_k^{vis} is constructed from the currently available perception evidence (line 4). Objects that remain unobserved are excluded from s_k^{vis} . In the kitchen domain (Fig. 2) the mug enters s_k^{vis} only when the container is opened. Segmentation masks collected across the simulator cameras determine the set V_k and also identifies the newly visible objects O_{new} since the previous planning event. The state constructor converts current evidence into symbolic relationships required for task planning (line 5). Though the object poses and region geometry are used internally to resolve these relationships, raw coordinates and region bounds are not exposed to the FM planner and remain available only to E_d for action grounding. The resulting representation can be viewed as a lightweight relational scene graph, and is reconstructed from current evidence at each k rather than maintained as a persistent geometric world model containing inferred states for unobserved objects.

In the RLBench/CoppeliaSim implementation, segmentation masks provide object-visibility evidence, while privileged simulator queries provide the geometric information used internally for semantic-region resolution and execution grounding. These backend-derived signals serve as proxies for the outputs of a reliable perception system, allowing the present evaluation to isolate planning, execution monitoring, and replanning behavior from perception errors.

Algorithm 1 ROBUST TAMP

Input: Goal g , domain d , initial environment state

Output: SUCCESS or FAILURE

```

1:  $h \leftarrow \emptyset$  // Action history initialized
2:  $e \leftarrow \emptyset$  // Event context initialized
3: while  $\neg$ task and replanning budget do
4:   Build  $s_k^{\text{vis}}$  from current evidence
5:    $c_k \leftarrow (s_k^{\text{vis}}, h_k, e_k, g)$ 
6:    $\alpha_k \leftarrow \text{Query}(c_k, g_\tau)$ 
7:    $P_d \leftarrow \text{Validate}(\alpha_k, \mathcal{A}_d, c_k)$ 
8:   if  $\neg P_d$  then
9:      $e \leftarrow$  structured planning failure
10:    continue
11:    $(h, O_{\text{new}}, \text{fail}_E) \leftarrow \text{Execute}(\alpha_k, E_d)$  // Action
    history  $h$ , new scene objects  $O_{\text{new}}$ , unrecovered execu-
    tion failure  $\text{fail}_E$ 
12:   if  $O_{\text{new}}$  then
13:      $e \leftarrow$  structured discovery event
14:     continue
15:   else if  $\text{fail}_E$  then
16:      $e \leftarrow$  structured failure event
17:     continue
18:   if  $G_\tau$  then
19:     return SUCCESS
20: return FAILURE

```

B. FM Planning and Layered Plan Validation

The `Query` subroutine takes in the visible context c_k and the natural-language goal g_τ , converting them into a structured prompt and α_k (line 6). During replanning, the prompt additionally includes h_k and structured context describing the event that triggered replanning. The text-only LLM planner receives this structured prompt, while the multimodal VLM planner additionally receives the newly captured camera views.

Before execution, a constrained parser validates whether the generated sequence uses supported actions, objects, and regions and satisfies structural action-sequence requirements (line 7). An invalid α_k is rejected and produces structured failure context for replanning (lines 8-10). The different failure modes considered are given in Table I.

Structural validation is intentionally separated from scene-dependent validation and the parser determines whether a sequence belongs to the executable interface. Current visibility, accessibility, and action preconditions are checked immediately before execution using the latest s_k^{vis} .

C. Heterogeneous Grounding and Execution

After α_k passes structural validation it is sent to the execution layer (line 11). α_k constrains the intended task order, while $\mathcal{X}_d$ determines how each action is physically realized. Robot motions such as approaching an object, manipulating an articulation, or retreating are therefore internal execution stages rather than model-generated actions.

E_d first identifies actions that can be realized as a bundle, e.g., a `pick(o)-place(o, r)` pair. Bundling allows $\mathcal{X}_d$ to

TABLE I: Taxonomy of failure modes for ROBUST TAMP.

Family	Failure condition	Layer	System response
Plan syntax	Unknown action token	L1	Reject plan
	Unknown object or region	L1	Reject plan
	Unknown lid object	L1	Reject plan
Plan logic	Orphan place	L1	Reject plan
	Pick-place mismatch	L1	Reject plan
	Missing post-pick place	L1	Reject plan
	Pick while holding object	L1	Reject plan
	Open while holding object	L1	Reject plan
Scene state	Newly visible object	L2	Replan
	Closed grill access	L2	Replan
	Closed box access	L2	Replan
	Lid or container blocked	L2	Replan
Motion execution	Empty pick trajectory	L2	Retry / replan
	Empty place trajectory	L2	Retry / replan
	No IK or motion solution	L2	Retry / replan
	Grasp failed	L2	Retry / replan
	Placement validation failed	L2	Retry / replan

Note. L1 denotes state-independent action-sequence validation before execution. L2 denotes failures detected through scene-state checks or low-level execution feedback during closed-loop execution. Motion execution occurs only when the plan is sent for execution.

preserve physical context across related actions and validate the outcome. Non-bundled actions are executed individually.

ROBUST TAMP does not require every action to use the same grounding mechanism. Scene-specific $\mathcal{X}_d$ may realize actions using PDDLStream-based motion planning [13], geometric helpers, or fixed motion primitives. PDDLStream framework samples continuous action parameters such as grasp poses and trajectories prior to planning and is employed for `pick` and `place` actions. Geometric and fixed motion primitives are used when an action is a structured manipulation sequence, e.g., `open` to open the container lid. Further, each $\mathcal{X}_d$ provides a standardized interface to the monitoring layer, reporting either successful execution or structured information explaining why execution could not be completed (lines 12-17). Consequently, the shared planning and replanning loop can operate over heterogeneous execution backends without requiring a common grounding procedure or uniform feasibility guarantees.

D. Closed-Loop Monitoring and Layered Failure Escalation

ROBUST TAMP monitors each action a_i or execution bundle before, during, and after execution. Before execution begins, scene-dependent checks determine whether the requested action is relevant with respect to s_k^{vis} . These checks capture conditions that cannot be established through structural plan validation alone, such as whether an object $o \in V_k$, whether a target region is accessible, or whether an articulation is obstructed. For each a_i , the corresponding $\mathcal{X}_d$ attempts to ground and realize it. When supported, $\mathcal{X}_d$ first handles recoverable failures through local fallback mechanisms, such as sampling alternative grasps, attempting alternative trajectories, or invoking a scene-specific recovery routine. These fallbacks remain internal to $\mathcal{X}_d$ and do not require another FM query.

Further to execution, the monitoring layer evaluates for the intended state change. For example, a transfer succeeds

only if the manipulated object is observed in the requested target region, while an articulation action succeeds only if the object’s configuration changes as intended.

If execution remains unsuccessful after the available fallbacks, the failure is converted into a structured event containing its source, execution stage, associated a_i , and supporting evidence. This is then fed back to the FM planner at the next planning event so that it can revise α_k . The resulting hierarchy rejects structural errors before execution, handles recoverable physical failures within the executor, and invokes the FM only when system-level replanning is required.

In the kitchen example, an attempt to open the container may be rejected before execution if a mug obstructs the lid. Alternatively, if a grocery transfer passes its preconditions but fails during manipulation, the executor first applies its available local fallbacks. Only an unrecovered failure is escalated to the planner with the structured context.

E. Discovery-Triggered Replanning

ROBUST TAMP treats the appearance of a previously unobserved scene object as a replanning event, even when the preceding execution routine completed successfully. After a_i completes via its $\mathcal{X}_d$, the monitoring layer refreshes s_k^{vis} . When new scene evidence is detected, the current execution sequence is interrupted at a stable execution horizon, and the pipeline reconstructs s_k^{vis} . In the kitchen example, opening the container is tagged as an articulation bundle and reveals the hidden mug. The pipeline rebuilds s_k^{vis} , where the mug enters the planner-visible state. The planner then generates a revised α_k incorporating the newly available object.

V. EXPERIMENTS AND EVALUATIONS

We evaluate ROBUST TAMP on three variants each of the kitchen and grill domains (Section V-A). The experiments are designed to investigate the following research questions:

- RQ1:** Can event-based replanning recover from hidden-object discovery in partially observable manipulation tasks?
- RQ2:** How do model modality and scale trade off task success, recovery quality, and planner-call latency?
- RQ3:** Does in-context learning (ICL) improve both explicit task completion and implicit non-target-object handling?
- RQ4:** Which failure mechanisms (planning, parsing, or execution-grounding failures) dominate unsuccessful runs?

A. Environments and Task Variants

We evaluate ROBUST TAMP on six curated RL-Bench/CoppeliaSim scenes divided between kitchen (Fig. 2) and grill (Fig. 1 A) domains. Within each domain, all variants use the same model-facing action interface and execution adapters; variants differ in their initial object arrangements, initially hidden objects, and required task procedures.

The kitchen variants K1–K3 share the natural-language goal of *placing all groceries in the cupboard and all mugs in the container*. In K1–K3, a mug is present on top of the container and must be relocated before the it can be opened. Upon opening the container, K1 reveals a previously hidden grocery item. K2 reveals both a grocery item that must be

TABLE II: Planning and replanning performance across the different task variants K1–K3 and G1–G3. Mean TSR averages across K1–K3 and G1–G3 and is reported separately from INH. Each variant column reports phone-excluded task-success rate for that model, modality, and prompting condition; G1 and G3 remove only the phone-to-table condition from task success. INH is computed only for G1 and G3 and measures whether the newly revealed phone is relocated to the table without being prompted in the natural-language goal.

Scale	Modality	Prompt	Planner	K1	K2	K3	G1	G2	G3	Mean TSR (%)	Mean PGC (%)	INH (%)
4B	LLM	Zero-shot	Qwen3-4B	70.0	90.0	0.0	0.0	0.0	0.0	26.7	42.5	0.0
	LLM	ICL	Qwen3-4B	80.0	80.0	80.0	100.0	100.0	100.0	90.0	97.3	0.0
	VLM	Zero-shot	Qwen3-VL-4B-Thinking	60.0	90.0	40.0	30.0	90.0	90.0	66.7	73.8	0.0
	VLM	ICL	Qwen3-VL-4B-Thinking	100.0	90.0	80.0	100.0	60.0	60.0	81.7	90.3	40.0
8B	LLM	Zero-shot	Qwen3-8B	80.0	60.0	80.0	100.0	100.0	100.0	86.7	91.8	0.0
	LLM	ICL	Qwen3-8B	80.0	100.0	40.0	100.0	100.0	100.0	86.7	93.5	100.0
	VLM	Zero-shot	Qwen3-VL-8B-Thinking	90.0	90.0	70.0	100.0	100.0	100.0	91.7	95.2	0.0
	VLM	ICL	Qwen3-VL-8B-Thinking	70.0	90.0	80.0	100.0	90.0	100.0	88.3	95.6	55.0
32B	LLM	Zero-shot	Qwen3-32B	90.0	100.0	80.0	100.0	100.0	100.0	95.0	98.6	0.0
	LLM	ICL	Qwen3-32B	90.0	70.0	70.0	100.0	100.0	100.0	88.3	95.7	100.0
	VLM	Zero-shot	Qwen3-VL-32B-Thinking	90.0	100.0	50.0	100.0	100.0	100.0	90.0	94.5	5.0
	VLM	ICL	Qwen3-VL-32B-Thinking	80.0	100.0	50.0	100.0	100.0	100.0	88.3	95.2	85.0

transferred and a mug that satisfies the goal. K3 adds a third mug on the table that must be placed in the container, increasing the plan horizon length relative to K1 and K2. These variants test whether replanning can incorporate newly revealed objects while preserving progress made before their discovery, thereby evaluating **RQ1**.

The natural-language command to grill variants G1–G3 is to *cook all raw meat using the grill and serve the cooked meat on the plate in the serving area*. Their initial configurations vary in the number and placement of task-relevant meat pieces. G1 and G3 also include a phone as a non-target object that the planner must handle based on the observed scene rather than as an explicit goal instruction. When the phone occupies the grill, the desired behavior is to relocate it to the table. G1 requires removing the phone before cooking and serving a meat piece initially located outside the grill. G2 reveals a meat piece already inside the grill and requires cooking and serving it together with two raw meat pieces present outside of it. G3 extends G2 by additionally having a phone inside the grill, producing the longest-horizon plan. These variants test whether the planner can infer appropriate treatment of non-target objects while satisfying a shared objective under different partially observable scene configurations, evaluating **RQ1**.

B. Compared Methods and Model Backends

To evaluate **RQ2** we compare against six FM models:

Size	LLM Planner	VLM Planner
4B / SMALL	Qwen3-4B	Qwen3-VL-4B-Thinking
8B / MEDIUM	Qwen3-8B	Qwen3-VL-8B-Thinking
32B / LARGE	Qwen3-32B	Qwen3-VL-32B-Thinking

All the models operate as interchangeable task-planning front-ends within the complete ROBUST TAMP pipeline. Each planner receives the same set of $c_k = (s_k^{\text{vis}}, h_k, e_k, g)$, and $\mathcal{A}_d$. VLM planners additionally receive a fresh stitched RGB composite captured from the simulator cameras at every initial-planning and replanning event. Each backend is evaluated under both zero-shot prompting and an ICL prompting condition (evaluates **RQ3**) using the same downstream execution stack. All generated plans are processed by

the same constrained parser, scene-dependent checks, monitoring mechanisms, deterministic task validators, and $\mathcal{X}_d$. Consequently, differences between model conditions reflect the task-planning front-end, input modality, and prompting condition rather than changes to downstream execution.

C. Evaluation Protocol and Metrics

Each model–variant–prompting condition is evaluated over 10 independent trials. Before each trial, the corresponding simulator scene is reset to its predefined initial configuration. To evaluate **RQ4**, all planners use the same set of deterministic task validators, $\mathcal{A}_d$, $\mathcal{X}_d$, monitoring mechanisms, and a replanning limit of 10. Planning is performed under both zero-shot and ICL prompting conditions using a temperature of 0 and a maximum generation length of 4096 tokens. All planner calls are served through a dedicated remote planner server, with models hosted on two NVIDIA RTX 4090 GPUs and implemented using the Hugging Face library.

Task Success Rate (TSR): We evaluate task completion using TSR, formally defined as

$$\text{Task Success} = \frac{1}{N} \sum_{k=1}^N \mathbb{I}[G_\tau(s_k^{\text{vis}}, h_k) = \text{true}]$$

where N is the number of planning events, $\mathbb{I}(\cdot)$ is the indicator function, and G_τ is the deterministic completion predicate for variant τ (Section III-C). For K1–K3, the predicate verifies the required final object-region relationships. For G1–G3, it additionally verifies the required temporally ordered cooking procedures from h_k . We report successful trials as both counts and percentages.

Partial Goal Completion (PGC): To measure partial success within each trial, we report PGC, defined as the proportion of deterministic goal conditions satisfied at termination:

$$\text{Partial Goal}_i = \frac{|\mathcal{R}_{\tau,i}^{\text{satisfied}}| + |\mathcal{P}_{\tau,i}^{\text{satisfied}}|}{|\mathcal{R}_\tau| + |\mathcal{P}_\tau|}$$

where $\mathcal{R}_{\tau,i}^{\text{satisfied}}$ contains the required final object–region relations satisfied at the end of trial i , and $\mathcal{P}_{\tau,i}^{\text{satisfied}}$ contains the required procedural predicates satisfied by h_k . As this metric uses the same atomic conditions as G_τ , it evaluates progress toward the task objective without requiring the executed

plan to match a canonical action sequence. A trial achieves complete task success exactly when $\text{Partial Goal}_i = 1$.

In-Context Learning (ICL): We evaluate whether providing scene-agnostic task examples improve replanning cycles—both positive and negative examples pertaining to valid object transfers, object interactions and execution recovery are provided. For example, in the context of opening an obstructed container lid, the action format provided is to *pick the blocking object and place it in a temporary region, followed by opening of the lid*.

Implicit Non-target Handling (INH): For G1 and G3, INH reports the proportion of trials in which the newly revealed phone is relocated to the table without being prompted in the natural-language goal or structured replanning context.

Replanning behavior: We characterize replanning behavior using the number of FM planner invocations and replanning cycles per episode. This is further separated into discovery-triggered replans—caused by the appearance of previously unobserved objects, and failure-triggered replans—caused by structural, pre-execution, runtime, or post-execution failures. This distinction prevents successful object-discovery events from being counted as execution failures.

Planner Cost: Total planner time is measured using cumulative planner-call latency per episode, mean planner-call latency per invocation, and end-to-end episode time from initial planning until termination.

Modes of failure: For failure analysis, every surfaced structured event is categorized by its identifier, detection layer, execution stage, source, and whether it requires FM replanning. We separately report structurally invalid plans rejected before execution ($\neg P_d$) and unrecovered execution failures escalated by the monitoring layer ($O_{new}, fail_E$). Recovery attempts by $\mathcal{X}_d$ (e.g., sampling alternative grasps) are not included in this quantitative analysis.

D. Results

Below, we present the evaluations corresponding to the research questions **RQ1 – RQ4** and discuss the findings.

Discovery-Triggered Replanning (RQ1). The mean TSR for 8B and 32B models is $> 86\%$ which demonstrates that ROBUST TAMP is able to successfully handle environments with partial observability. The mechanism of replanning upon discovery allows the model to incorporate the total set of objects required for task completion. For the smaller 4B models the mean TSR is $> 66\%$ for 3 out of 4 models. It is imperative to mention that binary task success understates partial progress. Even conditions with low mean TSR can satisfy a substantial fraction of the required final relations and procedural predicates before termination. Even the lowest performance on mean TSR of 26.7% from the 4B LLM zero-shot condition corresponded to a noticeably higher mean PGC of 42.5%, indicating that many failures occur after partial completion rather than at the initial planning stage.

Model Modality and Scale (RQ2). Fig. 3 illustrates the impact of model scale and modality on planning performance. Overall, larger models (8B and 32B) require fewer failure-triggered replans, indicating improved planning robustness,

but their planner-call latency is substantially higher. Notable, 8B models achieve TSR comparable to those of 32B models and are computationally cheaper. This suggests that medium-scale planners offer the most favorable tradeoff between success, recovery quality, and planning cost. It is also noticed that for both the 8B and 32B models, VLMs do not any additional advantage when the planner is provided with explicit object-region state $\text{region}(s_k^{\text{vis}}, o)$. Their TSR are generally comparable to text-only LLMs, and require higher planner-call latency as shown in Fig. 3.

ICL and INH (RQ3). Table II shows that TSR and INH differ in their success rates. In G1 and G3, zero-shot models often complete the stated cooking-and-serving task while leaving the newly revealed phone unhandled (both the 8B and 32B LLM zero-shot models achieve 100% TSR on G1 but 0% INH). ICL improves this behavior as the examples demonstrate the need for non-target object interactions. With ICL, both 8B and 32B LLMs achieve 100% INH. ICL also had large positive impact on TSR for the smaller models and substantially improved the executable task structure and grill-task success. For instance, the 4B LLM improves from 0% zero-shot TSR across all grill variants to 100% with ICL. In contrast, for larger models, changes to overall TSR are small, while the gain in INH is substantial. This illustrates that the models are too narrowly focused on the intended tasks and ignore the non-relevant objects even in the cases where ignoring such objects can have negative consequence.

Dominant Failures (RQ4). Fig. 4 shows the dominant failure mechanisms across model scales. Smaller models, especially the 4B variants, are dominated by parser and executable-interface failures, indicating difficulty in producing structurally valid action sequences. Larger models more consistently satisfy parser and task-interface constraints, with majority of the failures caused by execution grounding, motion, placement, and post-execution validation. Fig. 4 also demonstrates that using ICL significantly reduces the number of parser/interface failures for all the LLMs.

VI. CONCLUSION

We proposed ROBUST TAMP, a modular LLM/VLM-guided framework for reactive TAMP where unseen task-relevant and non-target objects may become visible during execution. Experiments show that the mechanism of discovery-triggered replanning enables the framework to integrate planning along with execution. This closed loop mechanism along with an efficient, hierarchical failure detection and recovery mechanism, enables ROBUST TAMP to effectively leverage the reasoning capabilities of modern LLMs and VLMS. Our analysis of the trade-offs between model modality, size, along with zero-shot and ICL prompting demonstrates that larger models do not necessarily result in better performance. Furthermore, the study highlights the need for a deeper analysis into how these models reason about implicit non-target objects.

REFERENCES

- [1] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,”

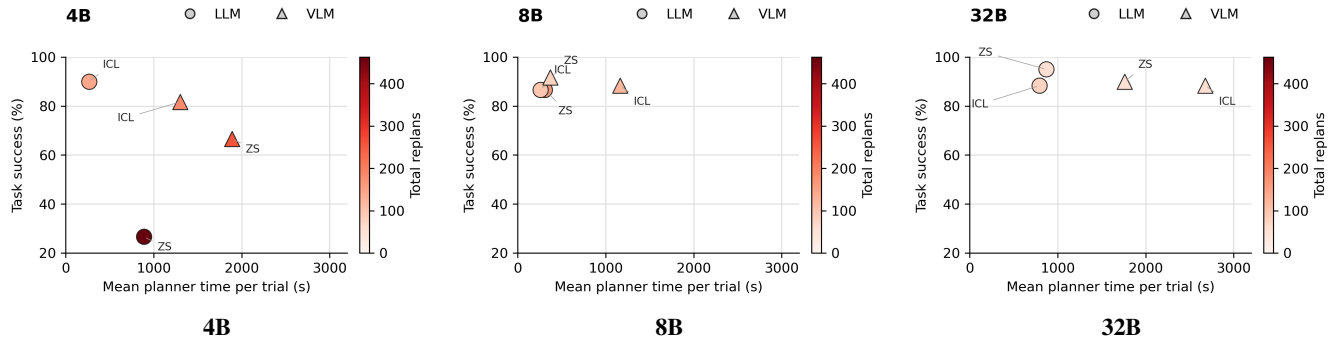


Fig. 3: Different evaluation metrics associated with K1–K3 and G1–G3, across 4B, 8B, and 32B model scales. Each point is one modality/prompt condition with the circular and triangular markers distinguishing the LLM and VLM planners. The x-axis shows mean planner time per trial, the y-axis shows TSR, and the color map encodes the total number of failure-triggered replans.

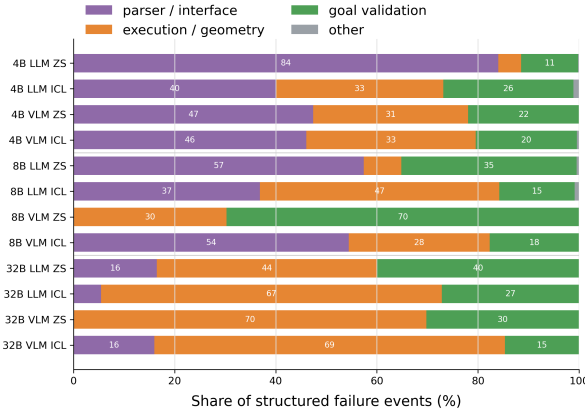


Fig. 4: Different modes of failures for the models

Annual review of control, robotics, and autonomous systems, vol. 4, no. 1, pp. 265–293, 2021.

- [2] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “Llm+p: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [3] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, “Autotamp: Autoregressive task and motion planning with llms as translators and checkers,” in *2024 IEEE International conference on robotics and automation (ICRA)*, pp. 6695–6702, 2024.
- [4] W. Guo, Z. Kingston, and L. E. Kavraki, “Castl: Constraints as specifications through llm translation for long-horizon task and motion planning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11957–11964, 2025.
- [5] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen, “Doremi: Grounding language model by detecting and recovering from plan-execution misalignment,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 12124–12131, 2024.
- [6] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling, “Trust the proc3s: Solving long-horizon robotics problems with llms and constraint satisfaction,” in *Conference on Robot Learning*, pp. 1362–1383, PMLR, 2025.
- [7] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu, “Llm3: Large language model-based task and motion planning with motion failure reasoning,” in *2024 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 12086–12092, 2024.
- [8] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett, “Open-world task and motion planning via vision-language model generated constraints,” *IEEE Robotics and Automation Letters*, vol. 11, no. 3, pp. 3366–3373, 2026.
- [9] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling, “Guiding long-horizon task and motion planning with vision language models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16847–16853, 2025.
- [10] M. Yan, M. Mengdibayev, A. Floros, W. Guo, L. E. Kavraki, and Z. Kingston, “Using vlm reasoning to constrain task and motion planning,” *arXiv preprint arXiv:2510.25548*, 2025.
- [11] S. Chen, A. Xiao, and D. Hsu, “Llm-state: Open world state representation for long-horizon task planning with large language model,” *arXiv preprint arXiv:2311.17406*, 2023.
- [12] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suen-derhauf, “Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning,” in *Conference on Robot Learning*, pp. 23–72, PMLR, 2023.
- [13] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, “Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the international conference on automated planning and scheduling*, vol. 30, pp. 440–448, 2020.
- [14] H. Guo, F. Wu, Y. Qin, R. Li, K. Li, and K. Li, “Recent trends in task and motion planning for robotics: A survey,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–36, 2023.
- [15] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox, “Robopoint: A vision-language model for spatial affordance prediction in robotics,” in *Conference on Robot Learning*, pp. 4005–4020, PMLR, 2025.
- [16] X. Zhang, Y. Ding, Y. Hayamizu, Z. Altaweel, Y. Zhu, Y. Zhu, P. Stone, C. Paxton, and S. Zhang, “Llm-grop: Visually grounded robot task and motion planning with large language models,” *The International Journal of Robotics Research*, p. 02783649251378196, 2025.
- [17] M. Merler, N. Dainese, M. Alakuijala, G. Bonetta, P. Ferrazzi, Y. Tian, B. Magnini, and P. Martinen, “Viplan: A benchmark for visual planning with symbolic predicates and vision-language models,” *arXiv preprint arXiv:2505.13180*, 2025.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Conference on Robot Learning*, pp. 1769–1782, PMLR, 2023.
- [19] S. S. Raman, V. Cohen, I. Idrees, E. Rosen, R. Mooney, S. Tellex, and D. Paulius, “Cape: Corrective actions from precondition errors using large language models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 14070–14077, 2024.
- [20] A. Mei, G.-N. Zhu, H. Zhang, and Z. Gan, “Replanvlm: Replanning robotic tasks with visual language models,” *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10201–10208, 2024.
- [21] Y. Feng, J. Han, Z. Yang, X. Yue, S. Levine, and J. Luo, “Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation,” in *Conference on Robot Learning*, pp. 2038–2062, PMLR, 2025.
- [22] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “Llm-planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 2998–3009, 2023.
- [23] Y. Liang, N. Kumar, H. Tang, A. Weller, J. B. Tenenbaum, T. Silver, J. F. Henriques, and K. Ellis, “Visualpredicator: Learning abstract world models with neuro-symbolic predicates for robot planning,” in *International Conference on Learning Representations*, vol. 2025, pp. 60416–60444, 2025.